

C++ - Module 01

Memory allocation, pointers to members,
references and switch statements

Summary:
This document contains the exercises of Module 01 from C++ modules.

Version: 11.1

13/9/26

Chapter IV

Exercise 00: BraiiiiiiinnnzzzZ

	Exercise: 00
	BraiiiiiiinnnzzzZ
Directory: *ex00/*	
Files to Submit: `Makefile, main.cpp, Zombie.{h, hpp}, Zombie.cpp, newZombie.cpp, randomChump.cpp`	
Forbidden: `None`	

First, implement a **Zombie** class. It has a private string attribute `name`.
Add a member function `void announce( void );` to the Zombie class. Zombies announce themselves as follows:

class Zombie ?

```
<name>: BraiiiiiiinnnzzzZ...
```

std::string name;
void announce(void);

Do not print the angle brackets (< and >). For a zombie named Foo, the message would be:

```
Foo: BraiiiiiiinnnzzzZ...
```

Then, implement the following two functions:

→ not mem func.

- `Zombie* newZombie( std::string name );`
  This function creates a zombie, names it, and returns it so you can use it outside of the function scope. → new use delete in main
- `void randomChump( std::string name );`
  This function creates a zombie, names it, and makes it announce itself.

Now, what is the actual point of the exercise? You have to determine in which case it is better to allocate zombies on the stack or the heap.

Zombies must be destroyed when you no longer need them. The destructor must print a message with the name of the zombie for debugging purposes.

* Initialization of a variable provides its initial value AT THE TIME OF construction
NEW[] can only use default constructor
new can use parametrized ctor

Chapter V

Exercise 01: Moar brainz!

	Exercise: 01
	Moar brainz!
Directory: *ex01/*	
Files to Submit: `Makefile, main.cpp, Zombie.{h, hpp}, Zombie.cpp, zombieHorde.cpp`	
Forbidden: `None`	

Time to create a **horde of Zombies**!

Implement the following function in the appropriate file:

rather: assignment
Num? *

```
Zombie* zombieHorde( int N, std::string name );
```

It must allocate `N` Zombie objects in a single allocation. Then, it must initialize the zombies, giving each of them the name passed as a parameter. The function returns a pointer to the first zombie.

in main? ← Implement your own tests to ensure that your `zombieHorde()` function works as expected. Try calling `announce()` for each of the zombies.

Do not forget to use `delete` to deallocate all the zombies and check for memory leaks.

1) allocate space for N zombies => 1 allocation ONE SINGLE
then 2) initialize + give the name
3) return pointer to the first zombie

Chapter VI

Exercise 02: HI THIS IS BRAIN

	Exercise: 02
	HI THIS IS BRAIN
Directory: *ex02/*	
Files to Submit: `Makefile, main.cpp`	
Forbidden: `None`	

contains:

Write a program that contains:

- A string variable initialized to 'HI THIS IS BRAIN'. std::string var("HI "),
- `stringPTR`: a pointer to the string. std::string * = &var
- `stringREF`: a reference to the string. std::string & = var

Your program must print:

prints maybe add something descriptive

std::cout

- The memory address of the string variable. <<&var
- The memory address held by `stringPTR`. << stringPTR
- The memory address held by `stringREF`. <<&stringREF

And then:

- The value of the string variable. <<var
- The value pointed to by `stringPTR`. << *stringPTR
- The value pointed to by `stringREF`. <<stringREF

That's all—no tricks. The goal of this exercise is to demystify references, which may seem completely new. Although there are some small differences, this is simply another syntax for something you already do: address manipulation.

reference - alias (alternative name for an existing variable)

- passing a Weapon by value doesn't work here because changing the original value wouldn't impact human object | passing by pointer requires change in main

Chapter VII

Exercise 03: Unnecessary violence

	Exercise: 03
	Unnecessary violence
Directory: ex03/	
Files to Submit: `Makefile, main.cpp, Weapon.{h, hpp}, Weapon.cpp, HumanA.{h, hpp}, HumanA.cpp, HumanB.{h, hpp}, HumanB.cpp`	
Forbidden: None	

Implement a **Weapon** class that has:

- A private attribute `type`, which is a string. std::string _type
- A `getType()` member function that returns a constant reference to `type`. const std::string& Weapon::getType
- A `setType()` member function that sets `type` using the new value passed as a parameter.

Now, create two classes: **HumanA** and **HumanB**. They both have a `Weapon` and a `name`. They also have a member function `attack()` that displays (without the angle brackets):

`<name> attacks with their <weapon type>`

HumanA and HumanB are almost identical except for these two small details:

- While **HumanA** takes the `Weapon` in its constructor, **HumanB** does not.
- **HumanB** may **not always** have a weapon, whereas **HumanA** will **always** be armed.

"nothing" → NULL

"default weapon" → parametrized ctor

If your implementation is correct, executing the following code will print an attack with "crude spiked club" followed by a second attack with "some other type of club" for both test cases:

copy initialization ←

```
int main()
{
  {
    Weapon  club = Weapon("crude spiked club");

    HumanA bob("Bob", club);
    bob.attack();
    club.setType("some other type of club");
    bob.attack();
  }
  {
    Weapon  club = Weapon("crude spiked club");

    HumanB jim("Jim");
    jim.setWeapon(club);
    jim.attack();
    club.setType("some other type of club");
    jim.attack();
  }

  return 0;
}
```

expected output for both:
first: crude spiked club
second: some other type of club

implement setWeapon for HumanB ←

Do not forget to check for **memory leaks**.

In which case do you think it would be best to use a pointer to Weapon? And a reference to Weapon? Why? Think about it before starting this exercise.

→ one needs to be a reference and other pointer

C++ class copy constructor [3] | Professor Hank Stalica
copy initialization syntax: T object = other; → delet.
T object(other);

> changes done to club by club.setType are modifying Weapon that human a and b already have

If one of them needs to have a pointer and the other reference it is obvious that the one with reference cannot change its state (weapon)
→ HumanA takes the Weapon in constructor and "can not leave it") => HumanA – Weapon&

modifying, modify, modified

perror → C function cerr → C++ stream
ifstream and ofstream don't guarantee to set errno

read: ifstream
write: ofstream
if <filename>.replace already exists → trunc

void open(const char * filename, ios_base::openmode mode

file is closed automatically, [RAII]
when object gets out of scope

Chapter VIII

Exercise 04: Sed is for losers

	Exercise: 04
	Sed is for losers
Directory:	ex04/
Files to Submit:	`Makefile, main.cpp, *.cpp, *.{h, hpp}`
Forbidden:	`std::string::replace`

Create a program that takes three parameters in the following order: a filename and two strings, `s1` and `s2`.

It must open the file `<filename>` and copy its content into a new file `<filename>.replace`, replacing every occurrence of `s1` with `s2`.

sed 's/s1/s2/g' filename

How can one define this? → don't read char by char

Using C file manipulation functions is forbidden and will be considered cheating. All the member functions of the class `std::string` are allowed, except `replace`. Use them wisely!

all − replace = allowed

Of course, handle unexpected inputs and errors. You must create and turn in your own tests to ensure that your program works as expected.

./sed_is_for_losers <filename> <s1> <s2>

replace every s1 with s2

Unexpeted imputs: file without premissions, empty, file that does not exist

s1 may be enewline

2001 space oddysey

Chapter IX

→ Karene - somebody that complains

Exercise 05: Harl 2.0

	Exercise: 05
	Harl 2.0
Directory:	ex05/
Files to Submit:	`Makefile, main.cpp, Harl.{h, hpp}, Harl.cpp`
Forbidden:	`None`

Do you know Harl? We all do, don't we? In case you don't, find below the kind of comments Harl makes. They are classified by levels:

- **'DEBUG'** level: Debug messages contain contextual information. They are mostly used for problem diagnosis.
  Example: *'I love having extra bacon for my 7XL-double-cheese-triple-pickle-special-ketchup burger. I really do!'*
- **'INFO'** level: These messages contain extensive information. They are helpful for tracing program execution in a production environment.
  Example: *'I cannot believe adding extra bacon costs more money. You didn't put enough bacon in my burger! If you did, I wouldn't be asking for more!'*
- **'WARNING'** level: Warning messages indicate a potential issue in the system. However, it can be handled or ignored.
  Example: *'I think I deserve to have some extra bacon for free. I've been coming for years, whereas you started working here just last month.'*
- **'ERROR'** level: These messages indicate that an unrecoverable error has occurred. This is usually a critical issue that requires manual intervention.
  Example: *'This is unacceptable! I want to speak to the manager now.'*

You are going to automate Harl. It won't be difficult since he always says the same things. You have to create a **Harl** class with the following private member functions:

- `void debug( void );`
- `void info( void );`
- `void warning( void );`
- `void error( void );`

Harl also has a public member function that calls the four member functions above depending on the level passed as a parameter:

```
void    complain( std::string level );
```

The goal of this exercise is to use pointers to member functions. This is not a suggestion. Harl has to complain without using a forest of if/else if/else. He doesn't think twice!

Create and turn in tests to show that Harl complains a lot. You can use the examples of comments listed above in the subject or choose to use comments of your own.

Chapter X

Exercise 06: Harl filter

	Exercise: 06
	Harl filter
Directory: *ex06/*	
Files to Submit: `Makefile, main.cpp, Harl.{h, hpp}, Harl.cpp`	
Forbidden: `None`	

Sometimes you don't want to pay attention to everything Harl says. Implement a system to filter what Harl says depending on the log levels you want to listen to.

Create a program that takes as a parameter one of the four levels. It will display all messages from this level and above. For example:

```
$> ./harlFilter "WARNING"
[ WARNING ]
I think I deserve to have some extra bacon for free.
I've been coming for years, whereas you started working here just last month.

[ ERROR ]
This is unacceptable! I want to speak to the manager now.

$> ./harlFilter "I am not sure how tired I am today..."
[ Probably complaining about insignificant problems ]
```

Although there are several ways to deal with Harl, one of the most effective is to SWITCH it off.

Give the name `harlFilter` to your executable.

You must use, and maybe discover, the switch statement in this exercise.



You can pass this module without doing exercise 06.

Why not inherit from std::string:

- std::string has no virtual destructor, so it is not designed as a base class.
- deleting a derived object through a std::string* is UB. the derived ctor is never called [typically]
- safe only if the derived class adds no data members and is never deleted via a base pointer [easy to break later]
- most string operations return plain std::string, so the derived type gets lost

Alternatives:

1. free function (ordinary)
2. composition - class with a std::string member